Capstone Demo Script and Viva Preparation

Live URL: https://travel-ai-agent-3llr.onrender.com

Total demo time: about 5 minutes. Keep to the times shown, and leave 1 to 2 minutes for questions.

Part 1: Live Demo Script (5 minutes)

Time	What to show	What to say
0:00 to 0:30	Open the live URL: https://travel-ai-agent-3llr.onrender.com	This is an AI travel planner built with
0:30 to 1:00	Home page: hero, destination cards, Your Trip Planner card	The home page lets users start from
1:00 to 2:00	Plan Trip: set From, To, dates. Open Advanced preferences. Click Plan my trip.	The AI agent uses 5 tools: flights, ho
2:00 to 2:30	Show the Markdown result: flight, hotel, weather, itinerary, budget	Every value comes from the tools, no
2:30 to 3:00	Click Download JSON. Open the file.	The same plan is available as structu
3:00 to 3:30	Itinerary tab: Goa, 3 days, Build itinerary	This tab uses local data only, so it do
3:30 to 4:00	Hotels tab (filter by stars) and Places tab (filter by type)	Browsing is free and fast, because it
4:00 to 4:30	Favorites (click heart) and My Trips (Save trip)	Users can save favorites and trips. T
4:30 to 5:00	Chat with AI button: ask one travel question. Then show the Reports folder.	The chat assistant answers general

Before the demo:

- Open the live URL 2 minutes early, so the Render free instance is awake.
- Check that Groq is not rate limited (one Plan my trip call should work).
- Keep a backup screenshot of the Plan result and the JSON file, in case the live site is slow.
- Have the GitHub repo open in another tab, to show the code structure if asked.

Part 2: Viva Questions and Answers

Short answers you can say in 20 to 30 seconds each.

Q1. What problem does this project solve?

Travellers spend a lot of time switching between flight, hotel, and attraction websites. The planner builds a complete, budget-aware plan from one request.

Q2. Why did you use an agent and not a single LLM prompt?

A single prompt cannot access live or structured data. The agent calls tools to get real data (flights, hotels, places, weather), so the answer is grounded.

Q3. How does the agent decide which tool to call?

The LLM reads the system prompt and the tool descriptions. It returns tool calls in its response, and the agent code runs them. This repeats until the LLM gives a final answer.

Q4. Why did you choose Groq and openai/gpt-oss-20b?

Groq is fast and has a free tier. The 20b model supports tool calling and is cheaper to run than the 120b model, which hit the rate limit faster during testing.

Q5. What is the role of temperature?

Temperature controls randomness. We used 0.2 so answers are consistent for the same request.

Q6. How do you prevent hallucinations?

The prompt says to use only tool data. Budget math is done by a tool, not by the model. We also check the results with an evaluation script.

Q7. How did you evaluate the agent?

We wrote 12 test queries and checked JSON validity, required sections, itinerary completeness, budget limit, and budget math. Results are in the Agent Performance Report.

Q8. What were the main problems you faced?

Tool-calling loops repeated the same tool, the itinerary had empty days when interests had few matches, and the free-tier token limit slowed testing. We fixed these with prompt rules, a fallback in the itinerary, and shorter tool outputs.

Q9. Why are the datasets sample data?

The project is a prototype. Real flight and hotel APIs need partner access or paid plans. We documented the limitations clearly, including unrealistic durations and synthetic names.

Q10. Why is the Itinerary tab separate from the agent?

The Itinerary tab uses local data only, so it is fast and free. The agent is used when the user needs a full plan with reasoning and live weather.

Q11. Why did you not use a database?

The data is small and read-only, so JSON files are enough. Favorites and trips use browser storage for the demo. A database is listed as future work.

Q12. Why FastAPI and not Flask or Django?

FastAPI is fast, uses type hints and Pydantic for validation, and generates API docs automatically at /docs.

Q13. How is the project deployed?

Code is on GitHub. Render builds it from requirements.txt and runs uvicorn with the port from the environment. The Groq key is stored as an environment variable.

Q14. What are the security concerns?

The API key is not in the code. User input is validated with Pydantic. Chat and plan responses are sanitized before showing them in the browser.

Q15. What would you improve next?

Real data sources, user accounts with a database, automated tests with pytest, and cost tracking for LLM calls.

Q16. What is the business value?

Lower support load for travel platforms, faster itinerary drafts for agencies, and clear budget comparisons for customers.

Part 3: Tips for the Presentation

- If a question is unclear, repeat it in your own words before answering.
- If you do not know an answer, say so, then explain how you would find out.
- Point to the project files when asked about implementation (api.py, agent/travel_agent.py, tools/).
- Mention the limitations yourself before the teacher asks. It shows honesty and understanding.

- Keep answers short. Offer to show the code or the test results if the teacher wants more detail.